Esteban Javier Antonio Alarcón Herrera

9941-16-13183

Programación III

Tarea VIII – Código Mejorado

Captura del código completo:

C-Free 5.0 - [C:\Users\javie\Desktop\Codigo Mejorado, Tarea VIII.cpp]

File Edit Search View Project Build Debug Tools Window Help

[D] mingw5

Codigo Corregido, Tarea VIII.cpp | Codigo Mejorado, Tarea VIII.cpp

- #include <iostream>
- std
- Nodo
  - dato
  - siguiente
  - anterior
- insertarInicio
- insertarFinal
- mostrarAdelante
- mostrarAtras
- buscar
- eliminar
- contarNodos
- liberarLista
- main

```cpp
#include <iostream>
using namespace std;

struct Nodo {
    int dato;
    Nodo* siguiente;
    Nodo* anterior;
};

//correcion aca por "Nodos"
void insertarInicio(Nodo*& head, Nodo*& tail, int valor) {
    Nodo* nuevo = new Nodo;
    nuevo->dato = valor;
    nuevo->siguiente = head;
    nuevo->anterior = NULL;

//correcion de que queremos igualar "=" no decir si es igual a "=="
    if (head != NULL) {
        head->anterior = nuevo;
    } else {
//Si la lista esta vacia, tail tambien sera el nuevo nodo
        tail = nuevo;
    }

    head = nuevo;
}

void insertarFinal(Nodo*& head, Nodo*& tail, int valor) {
    Nodo* nuevo = new Nodo;
    nuevo->dato = valor;
    nuevo->siguiente = NULL;
    nuevo->anterior = tail;

//aca la correcion es al contrario, queremos decir si es igual, no igualar a NULL
    if (head == NULL) {
        head = nuevo;
        tail = nuevo;
        return;
    }

```

C-Free 5.0 - [C:\Users\javie\Desktop\Codigo Mejorado, Tarea VIII.cpp]

File Edit Search View Project Build Debug Tools Window Help

[D] mingw5

Codigo Corregido, Tarea VIII.cpp | Codigo Mejorado, Tarea VIII.cpp

- #include <iostream>
- std
- Nodo
  - dato
  - siguiente
  - anterior
- insertarInicio
- insertarFinal
- mostrarAdelante
- mostrarAtras
- buscar
- eliminar
- contarNodos
- liberarLista
- main

```cpp

//Ya no recorremos toda la lista, usamos tail directamente
    tail->siguiente = nuevo;

//Al hacer que temp se convierta en el nuevo, debemos igualarlo aca
    tail = nuevo;
}

void mostrarAdelante(Nodo* head) {

//Otra vez debemos aclarar que si es igual y que no estamos igualando
    if (head == NULL) {
        cout << "Lista vacia\n";
        return;
    }

    Nodo* temp = head;

    cout << "\nLista hacia adelante: ";
    while (temp != NULL) {
        cout << temp->dato << " <-> ";

//Estamos haciendo la funcion hacia delante, por lo que no es anterior, si no que siguiente
        temp = temp->siguiente;
    }

    cout << "NULL\n";
}

void mostrarAtras(Nodo* tail) {
    if (tail == NULL) {
        cout << "Lista vacia\n";
        return;
    }

//Ahora iniciamos desde tail directamente
    Nodo* temp = tail;

    cout << "\nLista hacia atras: ";
    while (temp != NULL) {
```

C-Free 5.0 - [C:\Users\javie\Desktop\Codigo Mejorado, Tarea VIII.cpp]

File Edit Search View Project Build Debug Tools Window Help

[D] mingw5

Codigo Corregido, Tarea VIII.cpp | Codigo Mejorado, Tarea VIII.cpp

- #include <iostream>
- std
- Nodo
  - dato
  - siguiente
  - anterior
- insertarInicio
- insertarFinal
- mostrarAdelante
- mostrarAtras
- buscar
- eliminar
- contarNodos
- liberarLista
- main

```cpp
    while (temp != NULL) {
        cout << temp->dato << " <-> ";
        temp = temp->anterior;
    }

    cout << "NULL\n";
}

bool buscar(Nodo* head, int valor) {
    Nodo* temp = head;

    while (temp != NULL) {

//Otra vez correcion de igualacion
        if (temp->dato == valor) {
            return true;
        }

        temp = temp->siguiente;
    }

    return false;
}

void eliminar(Nodo*& head, Nodo*& tail, int valor) {
    if (head == NULL) {
        cout << "La lista esta vacia.\n";
        return;
    }

    Nodo* actual = head;

    while (actual != NULL && actual->dato != valor) {
        actual = actual->siguiente;
    }

    if (actual == NULL) {
        cout << "Dato no encontrado.\n";
        return;
    }
```

C-Free 5.0 - [C:\Users\javie\Desktop\Codigo Mejorado, Tarea VIII.cpp]

File Edit Search View Project Build Debug Tools Window Help

[D] mingw5

Codigo Corregido, Tarea VIII.cpp | Codigo Mejorado, Tarea VIII.cpp

- #include <iostream>
- std
- Nodo
  - dato
  - siguiente
  - anterior
- insertarInicio
- insertarFinal
- mostrarAdelante
- mostrarAtras
- buscar
- eliminar
- contarNodos
- liberarLista
- main

```cpp
    }

//Si solo existe un nodo
    if (head == tail && actual == head) {
        head = NULL;
        tail = NULL;
        delete actual;
        cout << "Nodo eliminado correctamente.\n";
        return;
    }

    if (actual == head) {
        head = head->siguiente;

//Se agrega una verificacion extra si es el primero
        if (head != NULL) {
            head->anterior = NULL;
        }

        delete actual;
        cout << "Nodo eliminado correctamente.\n";
        return;
    }

//Si se elimina el ultimo nodo
    if (actual == tail) {
        tail = tail->anterior;
        tail->siguiente = NULL;
        delete actual;
        cout << "Nodo eliminado correctamente.\n";
        return;
    }

    if (actual->siguiente != NULL) {

//correcion para apuntar al nodo anterior
        actual->siguiente->anterior = actual->anterior;
    }

    if (actual->anterior != NULL) {
```

C-Free 5.0 - [C:\Users\javie\Desktop\Codigo Mejorado, Tarea VIII.cpp]

File Edit Search View Project Build Debug Tools Window Help

[D] mingw5

Codigo Corregido, Tarea VIII.cpp | Codigo Mejorado, Tarea VIII.cpp

- #include <iostream>
- std
- Nodo
  - dato
  - siguiente
  - anterior
- insertarInicio
- insertarFinal
- mostrarAdelante
- mostrarAtras
- buscar
- eliminar
- contarNodos
- liberarLista
- main

```
    if (actual->anterior != NULL) {
        actual->anterior->siguiente = actual->siguiente;
    }

    delete actual;
    cout << "Nodo eliminado correctamente.\n";
}

int contarNodos(Nodo* head) {

//contador debe inicializarse en 0
    int contador = 0;
    Nodo* temp = head;

    while (temp != NULL) {
        contador++;
        temp = temp->siguiente;
    }

    return contador;
}

void liberarLista(Nodo*& head, Nodo*& tail) {

//punto y coma
    Nodo* temp = head;

    while (temp != NULL) {
        Nodo* aux = temp;

//Debe ir a siguiente, no anterior
        temp = temp->siguiente;
        delete aux;
    }

//Ambos punteros quedan vacios
    head = NULL;
    tail = NULL;
}
```

Build

C-Free 5.0 - [C:\Users\javie\Desktop\Codigo Mejorado, Tarea VIII.cpp]

File Edit Search View Project Build Debug Tools Window Help

[D] mingw5

Codigo Corregido, Tarea VIII.cpp | Codigo Mejorado, Tarea VIII.cpp

- #include <iostream>
- std
- Nodo
  - dato
  - siguiente
  - anterior
- insertarInicio
- insertarFinal
- mostrarAdelante
- mostrarAtras
- buscar
- eliminar
- contarNodos
- liberarLista
- main

```cpp

int main() {

//Lo iniciamos en NULL, otra correcion de igualacion
    Nodo* head = NULL;
    Nodo* tail = NULL;

    int opcion, valor;

    do {
        cout << "\n===== MENU =====\n";
        cout << "1. Insertar al inicio\n";
        cout << "2. Insertar al final\n";
        cout << "3. Mostrar hacia adelante\n";
        cout << "4. Mostrar hacia atras\n";
        cout << "5. Buscar dato\n";
        cout << "6. Eliminar dato\n";
        cout << "7. Contar nodos\n";
        cout << "8. Salir\n";
        cout << "Seleccione una opcion: ";
        cin >> opcion;

        switch(opcion) {
            case 1:
                cout << "Ingrese valor: ";
                cin >> valor;
                insertarInicio(head, tail, valor);
                cout << "Dato insertado al inicio.\n";
                break;

            case 2:
                cout << "Ingrese valor: ";
                cin >> valor;
                insertarFinal(head, tail, valor);
                cout << "Dato insertado al final.\n";
                break;

            case 3:
                mostrarAdelante(head);
                break;
```

C-Free 5.0 - [C:\Users\javie\Desktop\Codigo Mejorado, Tarea VIII.cpp]

File Edit Search View Project Build Debug Tools Window Help

[D] mingw5

Codigo Corregido, Tarea VIII.cpp | Codigo Mejorado, Tarea VIII.cpp

main

- #include <iostream>
- std
- Nodo
  - dato
  - siguiente
  - anterior
- insertarInicio
- insertarFinal
- mostrarAdelante
- mostrarAtras
- buscar
- eliminar
- contarNodos
- liberarLista
- main

```
235                 break;
236
237             case 4:
238                 mostrarAtras(tail);
239                 break;
240
241             case 5:
242                 cout << "Ingrese valor a buscar: ";
243                 cin >> valor;
244
245 //Correcion de igualacion
246                 if (buscar(head, valor) == true) {
247                     cout << "El valor SI existe en la lista.\n";
248                 } else {
249                     cout << "El valor NO existe en la lista.\n";
250                 }
251                 break;
252
253             case 6:
254                 cout << "Ingrese valor a eliminar: ";
255                 cin >> valor;
256                 eliminar(head, tail, valor);
257                 break;
258
259             case 7:
260                 cout << "Total de nodos: " << contarNodos(head) << endl;
261                 break;
262
263             case 8:
264                 cout << "Saliendo del programa...\n";
265                 break;
266
267             default:
268                 cout << "Opcion invalida.\n";
269         }
270
271 //Correccion simple de que solo hay 8 casos, no 20
272     } while (opcion != 8);
273
274     liberarLista(head, tail);
```

C-Free 5.0 - [C:\Users\javie\Desktop\Codigo Mejorado, Tarea VIII.cpp]

File Edit Search View Project Build Debug Tools Window Help

[D] mingw5

Codigo Corregido, Tarea VIII.cpp | Codigo Mejorado, Tarea VIII.cpp

```
250                 }
251                 break;
252
253             case 6:
254                 cout << "Ingrese valor a eliminar: ";
255                 cin >> valor;
256                 eliminar(head, tail, valor);
257                 break;
258
259             case 7:
260                 cout << "Total de nodos: " << contarNodos(head) << endl;
261                 break;
262
263             case 8:
264                 cout << "Saliendo del programa...\n";
265                 break;
266
267             default:
268                 cout << "Opcion invalida.\n";
269         }
270
271 //Correccion simple de que solo hay 8 casos, no 20
272     } while (opcion != 8);
273
274     liberarLista(head, tail);
275
276     return 0;
277 }
```

Captura de compilación exitosa:

Build

```
----------Configuration: mingw5 - CUI Debug, Builder Type: MinGW----------

Checking file dependency...

Complete Make Codigo Mejorado, Tarea VIII: 0 error(s), 0 warning(s)
Generated C:\Users\javie\Desktop\Codigo Mejorado, Tarea VIII.exe
```

Captura de ejecución:

"C:\Users\javie\Desktop\Codi

```
===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion: 1
Ingrese valor: 5
Dato insertado al inicio.

===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion: 2
Ingrese valor: 8
Dato insertado al final.

===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion: 3

Lista hacia adelante: 5 <-> 8 <-> NULL

===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion: 4
```

```
"C:\Users\javie\Desktop\Codi

Lista hacia atras: 8 <-> 5 <-> NULL

===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion: 5
Ingrese valor a buscar: 8
El valor SI existe en la lista.

===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion: 6
Ingrese valor a eliminar: 5
Nodo eliminado correctamente.

===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion: 7
Total de nodos: 1

===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion: 8
Saliendo del programa...
```

PREGUNTAS:

1. ¿Qué errores presentaba el código original?

   Presentaba varios errores, de compilación y lógica, entre los que más pude ver fueron malas igualaciones en varios de los “if” del programa, también al momento de declarar estructuras o inclusive una estructura de más que no estaba bien realizada, luego algunos punteros no se actualizaban de forma correcta.

2. ¿Qué problemas tenía la versión sin tail al recorrer hacia atrás?

   La versión sin tail tenía el problema de que para mostrar la lista hacia atraás primero debía recorrer toda la lista desde head hasta llegar al ultimo nodo, lo que hacia que cada vez que se quería imprimir para atrás se hacía un recorrido completo de más, lo cual no es óptimo.

3. ¿Qué ventaja aporta usar tail?

   La ventaja es que hay un acceso directo al último nodo y nos permite optimizar el programa.

4. ¿Qué operaciones mejoran con esta modificación?

   Serían la de insertar al final, la de la eliminación del ultimo nodo, o la de imprimir hacia atrás.

5. ¿Qué cuidados adicionales deben tenerse al eliminar nodos cuando existe tail?

   En general el cuidado que se tiene con el puntero de head, transferir por ejemplo tail al siguiente en caso de ser eliminado, o que no se pierda el acceso al mismo y actualizarlo cuando sea necesario.

Enlace al repositorio de GitHub